DAM DISEÑO DE INTERFACES UNIDAD 3

CheatSheet #10: Unidad 3 · Ejercicio 4 + Proyecto Definitivo

WPF + MahApps + MaterialDesign + navegación entre ventanas + login modal

IES Henri Matisse
2º DAM · Examen práctico

EJERCICIO 04 Menú Hamburguesa con HamburgerMenu de MahApps

MAH:HAMBURGERMENU

Concepto clave: el ejercicio trata de crear una navegación lateral en estilo móvil/desktop con `HamburgerMenu`, usando `ItemsSource` para las opciones principales y `OptionsItemsSource` para las secundarias. La clave está en personalizar la plantilla del menú.

Resumen: no es un simple menú; es una composición de `HamburgerMenu` + `DataTemplate` + `Grid` + `Icons` + `ItemsSource`. Eso permite que el mismo patrón sirva para menús, inventarios o administración.

1. Estructura visual
El menú se monta en un `Grid` y el control principal es `mah:HamburgerMenu`.

2. Plantilla reutilizable
`DataTemplate` define cómo dibujar cada ítem: icono + texto + altura uniforme.

3. Qué hace la lógica
En este caso la lógica es casi nula; el comportamiento visual es por XAML. Eso es típico del ejercicio.

XAML CLAVE — WPF

```
<!-- Plantilla del elemento del menú -->
<DataTemplate x:Key="MenuItemTemplateIcon" DataType="{x:Type mah:HamburgerM
    <Grid Height="48">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="48" />
            <ColumnDefinition />
        </Grid.ColumnDefinitions>
        <ContentControl Grid.Column="0" Content="{Binding Icon}" />
        <TextBlock Grid.Column="1" Text="{Binding Label}" />
    </Grid>
</DataTemplate>

<!-- Menú principal -->
<mah:HamburgerMenu
    PaneBackground="#3B3B3B"
    DisplayMode="CompactInline"
    ItemTemplate="{StaticResource MenuItemTemplateIcon}"
    OptionsItemTemplate="{StaticResource MenuItemTemplateIcon}">
    <mah:HamburgerMenu.ItemsSource>
        <mah:HamburgerMenuItemCollection>
            <mah:HamburgerMenuIconItem Label="Préstamos" Icon="{mat:PackIco
            <mah:HamburgerMenuIconItem Label="Administración" Icon="{mat:Pa
        </mah:HamburgerMenuItemCollection>
    </mah:HamburgerMenu.ItemsSource>
</mah:HamburgerMenu>
```

LÓGICA — C#

```
public partial class MainWindow : MetroWindow
{
    public MainWindow()
    {
        InitializeComponent();
    }
}
```

⚠ TRAMPAS DEL EJERCICIO 4

- Si no usas `mah:HamburgerMenu`, el menú no integra bien con MahApps.
- Los `HamburgerMenuIconItem` deben ir dentro de `HamburgerMenuItemCollection`.
- El `DataTemplate` debe usar `{Binding Icon}` y `{Binding Label}`.
- Si cambias la altura del título del icono, se rompe la alineación visual.

DAM DISEÑO DE INTERFACES PROYECTO

Proyecto Definitivo

Login modal, inyección de dependencias y navegación entre ventanas

IES Henri Matisse
2º DAM · Proyecto final

PROYECTO Login + navegación + cierre de sesión

PANTALLALOGIN + MAINWINDOW

Qué se está evaluando: la capacidad de combinar estilo visual, recursos globales y una navegación real entre ventanas. No se trata solo de "poner un formulario bonito"; hay que entender el flujo: login → abrir ventana principal → volver al login al cerrar sesión.

Regla de oro: el contenedor crea las ventanas; el login recibe `MainWindow` por constructor y la ventana principal recibe `Func<PantallaLogin>` para pedir un login nuevo al cerrar sesión. La fábrica diferida evita una dependencia circular inmediata.

1. App.xaml + App.xaml.cs
Los diccionarios visuales siguen en `App.xaml`; `App.xaml.cs` registra las ventanas y las resuelve al iniciar.

2. PantallaLogin.xaml
Conserva el diseño y recibe `MainWindow` como dependencia para abrirla tras validar los campos.

3. MainWindow.xaml
La pantalla de trabajo recibe una fábrica de login para cerrar sesión sin instanciar ventanas con `new`.

APP.XAML.CS · REGISTRO Y ARRANQUE — DI

```
// App.xaml conserva los diccionarios visuales.
      // Se elimina StartupUri: el contenedor inicia la ventana.
      private static void ConfigureServices(IServiceCollection services)
      {
        services.AddTransient<PantallaLogin>();
        services.AddTransient<MainWindow>();
        services.AddTransient<Func<PantallaLogin>>(sp =>
          () => sp.GetRequiredService<PantallaLogin>());
      }

      protected override void OnStartup(StartupEventArgs e)
      {
        base.OnStartup(e);
        var login = _serviceProvider.GetRequiredService<PantallaLogin>();
        MainWindow = login;
        ShutdownMode = ShutdownMode.OnMainWindowClose;
        login.Show();
      }
```

LOGIN C# — Evento

```
private readonly MainWindow _mainWindow;

      public PantallaLogin(MainWindow mainWindow)
      {
        InitializeComponent();
        _mainWindow = mainWindow;
      }

      private async void btnLogin_Click(object sender,
{
    if (string.IsNullOrEmpty(txtcorreo.Text) ||
        string.IsNullOrEmpty(passClaveAcceso.Password))
    {
        await this.ShowMessageAsync("Error de inicio de
            "Por favor, ingrese su correo y contraseña.
    }
    else
    {
      Application.Current.MainWindow = _mainWindow;
      _mainWindow.Show();
      Close();
    }
}
```

MAINWINDOW · FÁBRICA DE LOGIN Y CIERRE DE SESIÓN — Constructor injection

```
private readonly Func<PantallaLogin> _loginFactory;

      public MainWindow(Func<PantallaLogin> loginFactory)
      {
        InitializeComponent();
        _loginFactory = loginFactory;
      }

      private void btmcerrarsesion_Click(object sender, RoutedEventArgs e)
{
        PantallaLogin pantallaLogin = _loginFactory();
        Application.Current.MainWindow = pantallaLogin;
    pantallaLogin.Show();
        Close();
}
```

⚠ TRAMPAS DEL PROYECTO DEFINITIVO

- Si no validas correo y contraseña, el diálogo `ShowMessageAsync` debe lanzarse antes de abrir la siguiente ventana.
- Registra `Microsoft.Extensions.DependencyInjection` y las ventanas con `AddTransient`; no las crees con `new` fuera del contenedor.
- El `Image` no es un fondo "mágico"; si el recurso no existe, la imagen no cargará y la pantalla se ve rota.
- Elimina `StartupUri` si `OnStartup` resuelve el login; de otro modo podrías abrir dos ventanas iniciales.
- Asigna `Application.Current.MainWindow` a la siguiente ventana antes de cerrar la actual para evitar que WPF cierre la aplicación.
- Esta práctica solo valida que los campos no estén vacíos; no comprueba credenciales reales.